

Infrastructure Requirements - PR-DR Deployment

Control Node

Services Running:

1. Jenkins (CI/CD Automation)

Purpose: Orchestrates the entire deployment pipeline **What it does:** - Triggers automated deployments based on schedules or manual triggers - Manages the deployment workflow (PR → validate → DR) - Provides a web UI for monitoring deployment status - Stores deployment history and logs - Handles rollback operations if deployments fail - Sends notifications about deployment success/failure

2. Ansible (Deployment Orchestration)

Purpose: Executes actual deployment tasks on remote servers **What it does:** - Connects to PR/DR servers via SSH - Stops Java services before deployment - Backs up existing JAR files - Copies new JAR files to target servers - Starts services with proper configurations - Validates deployments by checking service health - Performs rollbacks by restoring backup JARs if needed - Manages configuration files (pr_vars.yml, dr_vars.yml)

3. Python 3.x

Purpose: Runtime for Ansible and custom scripts **What it does:** - Executes Ansible playbooks (Ansible is Python-based) - Runs automation scripts for pre/post-deployment tasks - Handles data processing and validation logic

4. SSH Client

Purpose: Secure remote access to PR/DR servers **What it does:** - Establishes encrypted connections to target servers - Authenticates using SSH keys (passwordless authentication) - Executes commands remotely during deployment

5. Git (Optional)

Purpose: Version control for deployment artifacts **What it does:** - Tracks changes to playbooks, scripts, and configurations - Pulls latest deployment code from repositories - Maintains deployment history

Files Stored:

- Playbooks (deploy, validate, rollback)

- Scripts (automation scripts)
- Configuration files (pr_vars.yml, dr_vars.yml)
- Inventory files (server lists)
- Jenkinsfile (pipeline definition)
- SSH keys

Resource Requirements:

- **Memory:** 8-16 GB recommended
 - **Disk:** 20-50 GB
 - **CPU:** 2-4 cores
-

PR Servers (Production/Primary)

Services Running:

1. Java Applications (Microservices)

Purpose: The actual business applications being deployed **What it does:** -
 Runs your microservices (JAR files) - Handles business logic and API requests
 - Processes transactions and data

2. JVM (Java Virtual Machine)

Purpose: Runtime environment for Java applications **What it does:** - Executes
 Java bytecode - Manages memory (heap: 4-8 GB) - Performs garbage
 collection (G1GC) - Optimizes application performance

3. Systemd (Service Management)

Purpose: Linux service manager **What it does:** - Starts/stops Java services -
 Manages service lifecycle (restart on failure) - Provides service status
 monitoring - Ensures services start on boot

External Connections:

- **PR Database server:** Persistent data storage (PostgreSQL)
- **PR Redis cache server:** In-memory caching for performance

Resource Requirements:

- **Memory:** 12-32 GB (8 GB for JVM heap)
- **Disk:** 20-100 GB
- **CPU:** 4-8 cores

JVM Configuration:

- Max Heap: 8 GB
- Min Heap: 4 GB
- GC Type: G1GC

DR Servers (Disaster Recovery)

Services Running:

1. Java Applications (Microservices)

Purpose: The actual business applications being deployed **What it does:** - Runs your microservices (JAR files) - Handles business logic and API requests - Processes transactions and data - Provides failover capability if PR environment fails

2. JVM (Java Virtual Machine)

Purpose: Runtime environment for Java applications **What it does:** - Executes Java bytecode - Manages memory (heap: 4-8 GB) - Performs garbage collection (G1GC) - Optimizes application performance

3. Systemd (Service Management)

Purpose: Linux service manager **What it does:** - Starts/stops Java services - Manages service lifecycle (restart on failure) - Provides service status monitoring - Ensures services start on boot

External Connections:

- **DR Database server:** Persistent data storage (PostgreSQL)
- **DR Redis cache server:** In-memory caching for performance

Resource Requirements:

- **Memory:** 12-32 GB (8 GB for JVM heap)
- **Disk:** 20-100 GB
- **CPU:** 4-8 cores

JVM Configuration:

- Max Heap: 8 GB
- Min Heap: 4 GB
- GC Type: G1GC

Separate Infrastructure (Not on these nodes)

1. PR Database Server (Separate Machine)

Purpose: Persistent data storage for Production environment **What it does:** - Stores application data (PostgreSQL) - Handles database queries from PR Java apps - Maintains data consistency and integrity - Provides ACID transactions

2. DR Database Server (Separate Machine)

Purpose: Persistent data storage for Disaster Recovery environment **What it does:** - Stores application data (PostgreSQL) - Handles database queries from DR Java apps - Maintains data consistency and integrity - Provides failover database capability

3. PR Redis Server (Separate Machine)

Purpose: In-memory caching for Production environment **What it does:** - Caches frequently accessed data - Improves application performance - Stores session data - Reduces database load

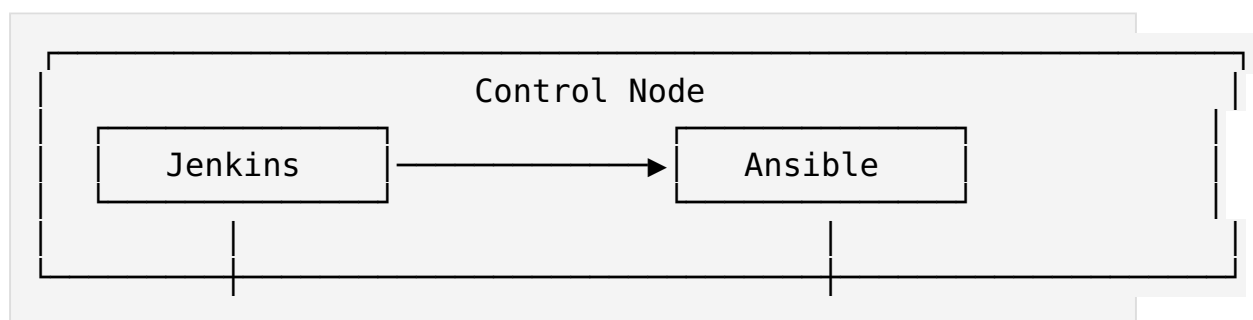
4. DR Redis Server (Separate Machine)

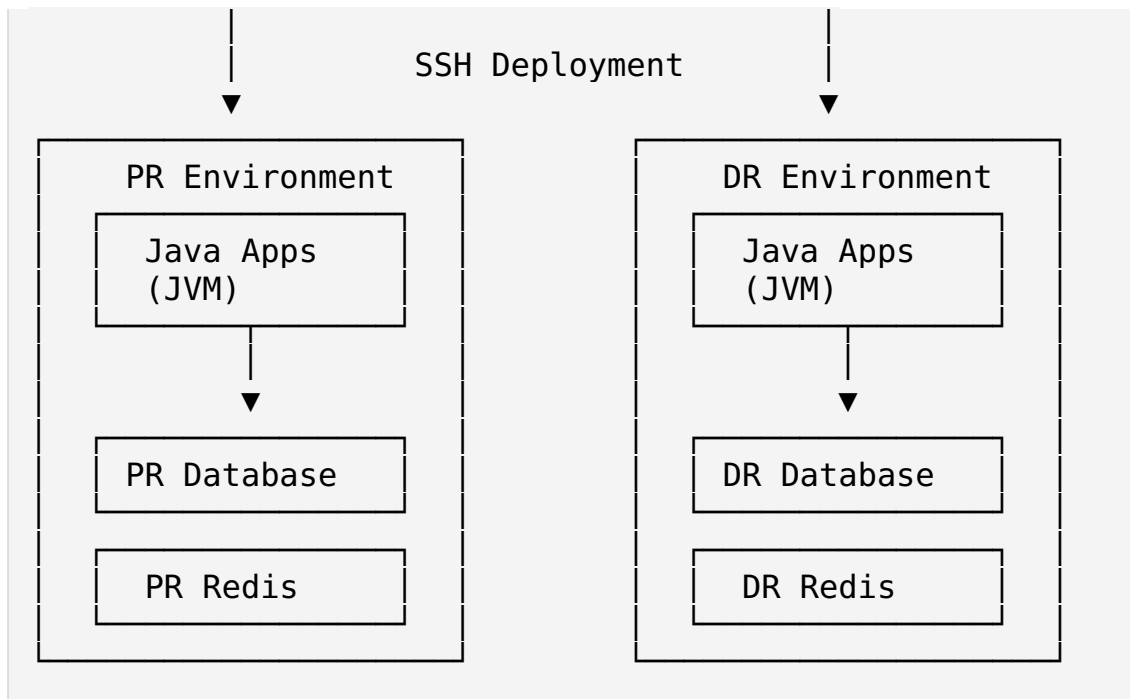
Purpose: In-memory caching for Disaster Recovery environment **What it does:** - Caches frequently accessed data - Improves application performance - Stores session data - Provides failover caching capability

Summary Table

Node	Services	Min RAM	Rec RAM	Min Disk	Rec Disk
Control Node	Jenkins + Ansible	8 GB	16 GB	20 GB	50 GB
PR Servers	Java Apps (microservices)	12 GB	16-32 GB	20 GB	50-100 GB
DR Servers	Java Apps (microservices)	12 GB	16-32 GB	20 GB	50-100 GB
Database Servers	PostgreSQL (separate)	16 GB	32+ GB	100 GB	500+ GB
Cache Servers	Redis (separate)	8 GB	16+ GB	20 GB	50+ GB

Architecture Diagram





Deployment Flow

1. **Jenkins** (Control Node) triggers deployment
 2. **Ansible** (Control Node) executes playbooks
 3. **Ansible** deploys to **PR Servers**
 4. **PR Servers** run Java applications
 5. **Ansible** validates PR deployment
 6. **Ansible** deploys to **DR Servers**
 7. **DR Servers** run Java applications
 8. **Jenkins** reports deployment status
-

Network Requirements

Control Node:

- SSH access to all PR servers (port 22)
- SSH access to all DR servers (port 22)
- Git access (port 443/22)

PR Servers:

- Access to PR Database (port 5432)
- Access to PR Redis (port 6379)
- Application ports (8080, 8081, 9090)

DR Servers:

- Access to DR Database (port 5432)
- Access to DR Redis (port 6379)

- Application ports (8080, 8081, 9090)
-

Minimum vs Recommended Specifications

For Testing:

- Control Node: 8 GB RAM, 20 GB disk
- PR/DR Servers: 12 GB RAM, 20 GB disk

For Production:

- Control Node: 16 GB RAM, 50 GB disk
 - PR/DR Servers: 32 GB RAM, 100 GB disk
 - Database Servers: 32+ GB RAM, 500+ GB disk
 - Redis Servers: 16+ GB RAM, 50+ GB disk
-

Why This Architecture?

Separation of Concerns:

- **Control Node** manages deployments (doesn't run business apps)
- **PR/DR servers** only run applications (no deployment tools)
- **Databases and caches** are isolated for performance and security

High Availability:

- **PR (Production)** and **DR (Disaster Recovery)** environments ensure business continuity
- If PR fails, DR can take over seamlessly

Automation:

- **Jenkins + Ansible** eliminates manual deployment errors
- Consistent, repeatable deployments
- Faster rollbacks in case of issues

Scalability:

- Each component can be scaled independently
 - Multiple PR/DR servers can be added to the inventory
-

Document Version: 1.0

Last Updated: May 2026

Location: /home/ubuntu/Automate-Patch-Release/docs/